```
    printf("area of circle=%lf\n",area);
    return 0;
}
```

To build the above program, we use the following command:

```
gcc simple.c -o simple
```

When you run the above command, have you ever thought of the various underlying phases of development? gcc and g++ (without options) undergo various phases like preprocessing, assembling, linking, etc, with the help of tools like *cpp*, *as*, *ld*, etc. So gcc and g++ act like wrappers for these tools.

To see each of these phases, let's try out some commands.

To see preprocessed output, you can use the *-E* option of *gcc* which invokes the *cpp* command internally. Here, you can see symbolic constants like PI replaced by their values; comments are removed, macros are expanded and header file contents are included.

```
gcc -E simple.c        #output comes on stdout by default
cpp simple.c -o simple.i
#you can use -o option to store output in a file, .i
extension is a convention to store outcome of preprocessor.
```

Optionally, we can provide symbolic constant PI externally using the *-D* option:

```
gcc -DPI=22.0/7.0 simple.c
```

To stop with generation of assembly code equivalent to source code the *-S* option is used, which internally uses *cc1*,*cc1plus* commands:

```
gcc -S simple.c        #generates simple.s
```

To locate the *cc1* command, use the following command:

```
gcc --print-prog-name=cc1 #path could be libexec/gcc/
mingw32/4.9.2/ in MingW
```

To generate assembly using *cc1*, use the command given below:

```
<path-of-cc1-dir>/cc1   simple.c
#cc1 is not located in bin dir of toolchain
$( gcc --print-prog-name) simple.c  #Linux specific
```

Optionally, you can generate the object file from the generated assembly code, as follows:

```
as simple.s -o simple.o
```

To stop compilation from the source code, use the code given below:

```
gcc -c simple.c        #generates object file simple.o
```

To combine one or more object files with the necessary library files, runtime support, and to generate executables using *collect2*, *ld* internally (e.g., printf is taken from the standard C library in the form of *libc.a* or *libc.so* and math functions are taken from libm.*), use the code given below:

```
gcc simple.o -o simple #generates executable/binary, a.out in
absence of -o option
```

To retain all intermediate files while building with *gcc*, type:

```
gcc --save-temps simple.c #keeps simple.i,simple.s,simple.o
```

gcc supports various optimisation levels by using options like *-O1*,*-O2*,*-O3* and *-O0* to turn off optimisation, and *-OS* for space optimising. If you are planning to debug generated code using *gdb*, the *-g<level>(-g1,-g2,-g3)* option can be used.

-g2 is assumed if *-g* is specified and *-g0* turns off debug support. You can use the *-I* option to specify the custom path of additional header files. Compile-specific options like *–I* and *-D* can go into the *CFLAGS* variable, and linker-specific options like *-L*, *-l* and *-static* can go into the *LDFLAGS* variable.

An example of a multi-file

Now let's try another example where the application is built from multiple source files; each source file with the included header files getting compiled individually is known as the translation unit. In this example, sum and square are invoked from the main function in *test.c*, and are defined in *sum.c*, *sqr.c* respectively. Assume the suitable function declarations and necessary header files.

```
#test.c:-
    int main()
    {
      int a,b,c,d;
      a=10,b=20;
      c=sum(a,b);
      d=square(a);
      printf("c=%d,d=%d\n",c,d);
      return 0;
    }
#sum.c:-
    int sum(int x,int y)
    {
      int z;
      z=x+y;
      return z;
    }
#sqr.c:-
```